Gate 03 — Auth & Access Control

Status: Complete **Preconditions:** Gate 02 complete **Unlocks:** Gate 04 (Django Admin), Gate 05 (DRF API) — both can start after this gate

---

Purpose

Implement the five-tier access model as a cross-cutting enforcement layer. Every subsequent gate's DRF endpoints and Admin configuration rely on the primitives built here. This gate produces: auth endpoints, a `TierPermission` DRF permission class, `for_tier()` queryset managers, institution-scoped access filtering, and session + token auth.

Deliverables

- `TierPermission(min_tier)` DRF permission class — gates endpoint access by `request.user.access_tier`
- `for_tier(tier)` queryset managers on `Species`, `ExSituPopulation`, `ConservationAssessment`
- Institution-scoped queryset filtering for Tier 3–4 users (can only edit records for their affiliated institution)
- Session authentication (Django Admin / web browser) and token authentication (DRF API)
- Registration flow: self-registration creates Tier 2 user with inactive account; email verification activates it
- Tier 3+ registration: admin must manually set `access_tier` via Django Admin (no self-promotion)
- Rate-limited login (5 attempts per 15 minutes per IP)

User Stories

BE-03-1: User Registration

As a researcher wanting to access occurrence data, **I want** to register with my email and create a Tier 2 account, **so that** I can view published datasets and field program reports.

DRF endpoint: `POST /api/v1/auth/register/` **Auth required:** None **Request body:**

```
{
  "email": "researcher@university.edu",
  "name": "Jane Smith",
  "password": "...",
  "institution_id": 42
}
```

Response: HTTP 201; sends verification email; account `is_active = False` until verified.

Acceptance Criteria:

Given a POST to `/api/v1/auth/register/` with a valid email and password ≥ 12 characters **When** the request is processed **Then** a User record is created with `access_tier = 2`, `is_active = False`, and a verification email is sent

Given a POST to `/api/v1/auth/register/` with a password under 12 characters **When** the request is processed **Then** HTTP 400 is returned with a password validation error; no User record is created

Given a POST to `/api/v1/auth/register/` with an email already registered **When** the request is processed **Then** HTTP 400 is returned; the existing account is not modified

---

BE-03-2: Email Verification

As a newly registered user, **I want** to verify my email address, **so that** my account becomes active and I can log in.

DRF endpoint: `GET /api/v1/auth/verify/?token=<token>` **Auth required:** None

Acceptance Criteria:

Given a valid verification token in the URL **When** the endpoint is called **Then** `User.is_active` is set to True and HTTP 200 is returned

Given an expired or invalid token **When** the endpoint is called **Then** HTTP 400 is returned; the account remains inactive

---

BE-03-3: Login

As an active user, **I want** to obtain an API token by logging in with my email and password, **so that** I can authenticate DRF API requests.

DRF endpoint: `POST /api/v1/auth/login/` **Auth required:** None **Request body:** `{"email": "...", "password": "..."}` **Response:** `{"token": "<token>", "access_tier": 2}`

Acceptance Criteria:

Given valid credentials for an active Tier 2 account **When** `POST /api/v1/auth/login/` is called **Then** HTTP 200 returns a DRF token and the user's `access_tier`

Given valid credentials for an inactive account (not yet verified) **When** `POST /api/v1/auth/login/` is called **Then** HTTP 403 is returned with a message indicating the account is not yet verified

Given 5 failed login attempts from the same IP within 15 minutes **When** a sixth attempt is made **Then** HTTP 429 is returned; the account is not locked (rate limit is IP-based, not account-based)

BE-03-4: Logout

DRF endpoint: `POST /api/v1/auth/logout/` **Auth required:** Token

Acceptance Criteria:

Given a valid token **When** `POST /api/v1/auth/logout/` is called **Then** the token is invalidated and subsequent requests with that token return HTTP 401

BE-03-5: Current User Profile

DRF endpoint: `GET /api/v1/auth/me/` **Auth required:** Token

Response:

```
{
  "id": 1,
  "email": "researcher@university.edu",
  "name": "Jane Smith",
  "access_tier": 2,
  "institution": {"id": 42, "name": "University of Antananarivo"}
}
```

Acceptance Criteria:

Given a valid Tier 3 token **When** `GET /api/v1/auth/me/` is called **Then** HTTP 200 returns the user's profile including `access_tier = 3` and affiliated institution

Given an unauthenticated request **When** `GET /api/v1/auth/me/` is called **Then** HTTP 401 is returned

---

BE-03-6: Tier Enforcement on Protected Endpoints

This story verifies the `TierPermission` class, not a specific endpoint — it is exercised by Gate 05 endpoints. The permission class is built here.

Acceptance Criteria:

Given an anonymous request (no token) **When** any endpoint with `permission_classes = [TierPermission(min_tier=2)]` is called **Then** HTTP 401 is returned

Given a Tier 2 token **When** an endpoint with `permission_classes = [TierPermission(min_tier=3)]` is called **Then** HTTP 403 is returned; the response body does not reveal what data exists at that tier

Given a Tier 3 token for a user affiliated with Institution A **When** a PATCH request to modify an ExSituPopulation record belonging to Institution B is made **Then** HTTP 403 is returned (institution-scoped enforcement)

Technical Implementation Notes

`TierPermission` class:

```python
class TierPermission(BasePermission):
    def __init__(self, min_tier: int):
        self.min_tier = min_tier

    def has_permission(self, request, view):
        if not request.user.is_authenticated:
            return False
        return request.user.access_tier >= self.min_tier
```

Use `functools.partial` or a factory function to allow `permission_classes = [TierPermission.require(3)]` in views.

`for_tier()` queryset managers:

- `Species.objects.for_tier(tier)` — no filtering at species level (all species are public); this manager exists for consistency and future use

- `ExSituPopulation.objects.for_tier(tier)` — returns all records for Tier 3+; returns empty queryset for Tier 1–2 (aggregate stats come from the dashboard endpoint, not this queryset)
- `ConservationAssessment.objects.for_tier(tier)` — Tier 1–2 sees only `review_status='accepted'` records; Tier 3+ sees all

Institution-scoped writes: Add `get_queryset()` override on Tier 3–4 viewsets that filters writable records to `institution = request.user.institution`. Read access is broader (Tier 3 can view all populations); write access is scoped to affiliated institution.

Auth backends:

- `SessionAuthentication` for Django Admin
- `TokenAuthentication` (DRF built-in) for API — use `rest_framework.authtoken`
- Do not use JWT for MVP (adds complexity without benefit at this scale)

Rate limiting: Use `django-ratelimit` or Django's built-in cache-based rate limiting on the login endpoint. Do not use third-party auth services.

Technical Tasks

- Implement `TierPermission` in `accounts/permissions.py` with unit tests covering all tier combinations
- Implement `for_tier()` queryset managers on `Species`, `ExSituPopulation`, `ConservationAssessment` managers with unit tests
- Implement institution-scoped queryset filtering utility in `accounts/scoping.py`
- Create `AuthViewSet` with register, verify, login, logout, me actions
- Configure `DEFAULT_AUTHENTICATION_CLASSES` in DRF settings
- Configure login rate limiting
- Write unit tests: registration flow, email verification, login success/failure, tier enforcement matrix (5x5), institution scoping
- Configure `django.contrib.auth` email backend for development (console backend); production email backend via settings

---

Out of Scope

- Django Admin configuration (Gate 04)
- Any data endpoint serializers (Gate 05)
- Tier 3+ self-registration UI (coordinators are manually elevated by admin)
- SAML/OIDC/SSO (post-MVP)
- Password reset flow (implement if time permits, not MVP-blocking)

Gate Exit Criteria

Before marking Gate 03 complete:

1. All auth endpoints return correct HTTP status codes per acceptance criteria
2. `TierPermission` unit tests cover all tier combinations (anonymous, Tier 1–5 against min_tier 1–5)
3. Institution-scoped queryset tests pass
4. Invoke **@test-writer** to write adversarial auth tests (token replay, tier escalation attempts, account enumeration via registration endpoint)
5. Invoke **@security-reviewer** — this gate touches auth, permissions, and credential handling

6. Invoke **@code-quality-reviewer** on all auth code

Reconciliation

Gate 03 reconciliation completed 2026-04-16. Full report: gate-03-reconciliation.md

Key deviations from spec (all improvements from security/code-quality reviews):

- Email verification changed from GET to POST (security: link prefetchers can trigger GET)
- Inactive user login returns 401 not 403 (security: prevents account enumeration)
- TierPermission implemented as factory function returning a class (DRF compatibility)
- Rate limiter uses atomic cache.add/cache.incr (code quality: TOCTOU fix)
- Verification URL points to FRONTEND_BASE_URL, not API endpoint
- scope_to_institution rejects users below Tier 3

Deferred to post-MVP: token expiry, password reset flow, AuditLog write path.